LangChain
Cheat Sheet

Every core building block, with the code you actually type, in one scroll.

NARESH EDAGOTTI

PRACTICAI

What LangChain Actually Is

BASICS

A standard interface over models, prompts, retrievers and tools. You code against the interface, so swapping a provider or a vector store is a one line change instead of a rewrite.

THE MENTAL MODEL

Components › Composed with LCEL › One Runnable › Your App

THE PACKAGE LAYOUT, KNOW THIS FIRST

- langchain-core has the interfaces, no heavy deps
- langchain has the chains, agents and retrieval logic
- langchain-openai and friends hold provider code
- langchain-community holds the long tail of integrations

> Most import errors come from guessing the package. When a class moves, it moved to one of these four.

Install and First Call START

WHAT THIS COVERS

Install only what you use. Keys live in the environment, never in code, because LangChain reads them automatically when the client is created.

```
# base plus one provider
pip install langchain langchain-openai python-dotenv

# add pieces as you need them
pip install langchain-community langchain-chroma pypdf
```

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()                 # reads .env into os.environ

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
print(llm.invoke("Say hello in five words").content)
```

YOUR .ENV FILE

- OPENAI_API_KEY for the model provider
- LANGSMITH_TRACING and LANGSMITH_API_KEY for tracing

One Kit to Clear Your AI Interview KIT

After seeing how AI interviews are evolving, I created an **AI Interview Kit.** It covers the areas companies actually test:

- AI fundamentals
- AI agents
- Transformers in depth
- Memory systems
- LLM architectures
- Databases
- Prompt engineering
- Backend for AI systems
- Fine tuning
- Deployment strategies
- RAG systems
- GenAI system design

Everything structured to help you prepare.

↓ Check the caption to enroll

Chat Models THE ENGINE

WHAT THIS IS

Every chat model exposes the same three methods. Learn them once and they work for every provider and every chain you ever build.

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,         # 0 factual, 1 creative
    max_tokens=512,
    timeout=30,
)

llm.invoke("one question")          # single call
llm.batch(["q1", "q2", "q3"])       # parallel calls

for chunk in llm.stream("write a haiku"):
    print(chunk.content, end="")  # token by token
```

READING THE CODE

batch is genuinely parallel, not a loop over invoke

Swap the class to change provider, the rest stays identical

Messages and Roles INPUT

WHAT THIS IS

Chat models do not take strings, they take a list of typed messages. The role tells the model who is speaking, which is how instructions outrank user input.

```python
from langchain_core.messages import (
    SystemMessage, HumanMessage, AIMessage)

msgs = [
    SystemMessage("You are a terse Python tutor."),
    HumanMessage("What is a list comprehension?"),
    AIMessage("A one line way to build a list."),
    HumanMessage("Show me one."),
]

res = llm.invoke(msgs)
print(res.content)
print(res.usage_metadata)   # token counts and cost
```

THE FOUR ROLES YOU WILL MEET

- System sets behaviour
- Human is the user turn
- AI is the model reply
- Tool returns a result

Prompt Templates REUSE

WHAT THIS IS

Stop building prompts with f strings. A template declares its variables, validates them at call time and slots cleanly into any chain.

```
from langchain_core.prompts import (
    ChatPromptTemplate, MessagesPlaceholder)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You translate into {language}."),
    MessagesPlaceholder("history", optional=True),
    ("human", "{text}"),
])

print(prompt.input_variables)   # language, text

# lock a variable early, fill the rest later
telugu = prompt.partial(language="Telugu")
telugu.invoke({"text": "good morning"})
```

READING THE CODE

- MessagesPlaceholder is where past turns get injected later
- partial pins values once so callers pass fewer arguments

Output Parsers STRUCTURE

WHAT THIS IS

Model output is text. Parsers turn it into the type your code expects, so nothing downstream has to do string surgery on a reply.

```python
from langchain_core.output_parsers import StrOutputParser
from pydantic import BaseModel, Field

# 1. plain text out of the AIMessage
chain = prompt | llm | StrOutputParser()

# 2. validated objects, the production choice
class Review(BaseModel):
    sentiment: str = Field(description="positive or negative")
    score: int = Field(description="1 to 5")
    topics: list[str]

structured = llm.with_structured_output(Review)
out = structured.invoke("The battery died in two hours.")
print(out.score, out.topics)   # a real object
```

> with_structured_output uses the provider tool calling API under the hood. It is far more reliable than asking for JSON in the prompt.

LCEL, the Pipe Operator CORE IDEA

WHAT THIS IS

LangChain Expression Language. Every component is a Runnable, and the pipe operator joins them into one bigger Runnable with the exact same interface.

```
chain = prompt | llm | StrOutputParser()

# the composed chain inherits everything for free
chain.invoke({"language": "Hindi", "text": "hello"})
chain.batch([{...}, {...}])       # parallel
chain.stream({...})               # token stream
await chain.ainvoke({...})        # async

print(chain.get_graph().draw_ascii())  # inspect it
```

WHY THIS MATTERS

- Any Runnable can pipe into any other Runnable
- Streaming, async, batching and retries come built in
- Every step shows up separately in your traces

Runnable Primitives GLUE

WHAT THIS IS

Three helpers cover almost every wiring problem. They let you branch, carry values forward and drop plain Python into the middle of a chain.

```python
from langchain_core.runnables import (
    RunnableParallel, RunnablePassthrough, RunnableLambda)

# run two branches at the same time
fan_out = RunnableParallel(
    summary=summary_chain,
    keywords=keyword_chain,
)

# keep the original input and add a new key
enriched = RunnablePassthrough.assign(
    context=lambda x: retriever.invoke(x["question"]))

# any function becomes a chain step
clean = RunnableLambda(lambda t: t.strip().lower())

chain = enriched | prompt | llm | StrOutputParser()
```

> assign is the one to memorise. It is how RAG chains carry the question forward while adding retrieved context beside it.

Loaders and Splitters INGEST

WHAT THIS IS

Loaders turn any source into Document objects with text and metadata. Splitters cut those into chunks small enough to embed and cheap enough to send.

```python
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, CSVLoader, TextLoader)
from langchain_text_splitters import (
    RecursiveCharacterTextSplitter)

docs = PyPDFLoader("policy.pdf").load()
print(docs[0].page_content, docs[0].metadata)

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
    add_start_index=True,
)
chunks = splitter.split_documents(docs)
```

QUICK RULES

- 400 to 800 chars for prose
- 10 to 20 percent overlap

Embeddings and Vector Stores

MEMORY

WHAT THIS IS

Embeddings turn text into vectors that carry meaning. The vector store indexes them so similar text can be found in milliseconds.

```python
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

emb = OpenAIEmbeddings(model="text-embedding-3-small")

store = Chroma.from_documents(
    documents=chunks,
    embedding=emb,
    persist_directory="./chroma_db",
)

store.similarity_search("refund window", k=4)
store.similarity_search_with_score("refund", k=4)
store.add_documents(new_chunks)   # incremental
```

RULES THAT SAVE YOU PAIN

- Same embedding model for indexing and querying, always
- Changing the model means reindexing everything

Retrievers ADVANCED

WHAT THIS IS

A retriever is any Runnable that takes a query and returns Documents. Wrapping one in another is how you upgrade search quality without touching your index.

```
base = store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 8, "fetch_k": 30})

# LLM writes several versions of the query
from langchain.retrievers.multi_query import MultiQueryRetriever
mq = MultiQueryRetriever.from_llm(retriever=base, llm=llm)

# keyword plus vector, the hybrid default
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever
hybrid = EnsembleRetriever(
    retrievers=[BM25Retriever.from_documents(chunks), base],
    weights=[0.4, 0.6])
```

WHEN TO REACH FOR EACH

- MMR kills near duplicates
- # BM25 catches exact terms

A RAG Chain in One Page PATTERN

WHAT THIS IS

The single most used LangChain pattern. Retrieve context, stuff it into a prompt, generate a grounded answer, keep the sources.

```
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import (
    create_stuff_documents_chain)

qa_prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer only from the context.\n\n{context}"),
    ("human", "{input}"),
])

combine = create_stuff_documents_chain(llm, qa_prompt)
rag = create_retrieval_chain(hybrid, combine)

out = rag.invoke({"input": "What is the refund window?"})
print(out["answer"])
print([d.metadata for d in out["context"]])  # sources
```

> The context key is filled by the retriever automatically. That is the only piece of magic in this pattern.

Chat History MULTI TURN

WHAT THIS IS

Chains are stateless by default. Wrap one in a history runnable and every call for a given session id gets the past turns injected automatically.

```python
from langchain_core.runnables.history import (
    RunnableWithMessageHistory)
from langchain_core.chat_history import (
    InMemoryChatMessageHistory)

sessions = {}
def get_history(session_id: str):
    return sessions.setdefault(
        session_id, InMemoryChatMessageHistory())

conv = RunnableWithMessageHistory(
    chain, get_history,
    input_messages_key="text",
    history_messages_key="history")

cfg = {"configurable": {"session_id": "user-42"}}
conv.invoke({"text": "and translate that too"}, cfg)
```

AT SCALE

- Back it with Redis or SQL
- trim_messages caps tokens

Tools and Tool Calling ACTIONS

WHAT THIS IS

A tool is a Python function the model is allowed to request. The model never runs it, it returns a structured request and your code decides whether to execute.

```python
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Return the current weather for a city."""
    return f"{city}: 31C and clear"

llm_tools = llm.bind_tools([get_weather])
res = llm_tools.invoke("Weather in Hyderabad?")

print(res.tool_calls)
# [{'name': 'get_weather', 'args': {'city': 'Hyderabad'}, ...}]

for call in res.tool_calls:
    result = get_weather.invoke(call["args"])
```

> The docstring is not documentation, it is the tool description the model reads. A vague docstring is a tool the model calls at the wrong time.

Agents AUTONOMY

WHAT THIS IS

An agent runs the tool loop for you. It calls the model, executes any requested tool, feeds the result back and repeats until the model returns a final answer.

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(
    model=llm,
    tools=[get_weather, search_docs],
    prompt="You are a concise research assistant.",
)

out = agent.invoke({"messages": [
    ("user", "Weather in Hyderabad and our leave policy?")]})

print(out["messages"][-1].content)
```

READING THE CODE

- The loop of think, act and observe runs until no tool is requested
- Agents now live in LangGraph, the old AgentExecutor is legacy
- Always cap iterations, a stuck loop burns real money

Streaming and Async SPEED

WHAT THIS IS

Perceived speed is what users judge. Streaming sends tokens as they are produced, and async lets one server handle many waiting requests at once.

```
# token stream, what you pipe into a UI
async for chunk in chain.astream({"text": q}):
    print(chunk, end="", flush=True)

# event stream, see every internal step
async for ev in chain.astream_events({"text": q}, version="v2"):
    if ev["event"] == "on_retriever_end":
        print("docs found:", len(ev["data"]["output"]))
    if ev["event"] == "on_chat_model_stream":
        print(ev["data"]["chunk"].content, end="")
```

> astream_events is the hook for showing progress messages like searching documents while the answer is still being written.

Reliability and Tracing SHIP IT

WHAT THIS IS

Providers rate limit, time out and go down. These four one liners are the difference between a demo and something you leave running.

```
safe = (llm
    .with_retry(stop_after_attempt=3)
    .with_fallbacks([backup_llm])
    .with_config({"tags": ["prod"], "run_name": "answer"}))

# cache identical calls, huge saving during dev
from langchain_core.globals import set_llm_cache
from langchain_community.cache import SQLiteCache
set_llm_cache(SQLiteCache(".langchain.db"))

# tracing, just two env vars
# LANGSMITH_TRACING=true
# LANGSMITH_API_KEY=ls__...
```

THE PRODUCTION CHECKLIST

- Retries and fallbacks
- Cache repeated calls
- Trace every run
- Log token usage

PRACTICAI

The Whole Cheat Sheet

- Chat models
- Messages
- Prompts
- Parsers
- LCEL pipes
- Runnables
- Loaders
- Vector stores
- Retrievers
- Tools and agents

Learn the Runnable interface once. Every other piece of LangChain plugs into it.

Save this for your next build. Follow for more.